CLASS RELATIONSHIPS

Table of Contents

1. Prerequisites....4

2. Purpose 4

3. Introduction....4

4. Detailed Explanation....4

4.1 Class Relationship....4

4.1.1 Relating with Real World Scenario 4

4.2 Class Relationship Types....7

4.2.1 Composition....7

4.2.1.1 Try It Yourself....8

4.2.1.2 UML Representation....15

4.2.2 Aggregation....15

4.2.2.1 Try It Yourself....16

4.2.2.2 UML Representation....20

4.2.3 Association 20

4.2.3.1 Try It Yourself....21

4.2.3.2 UML Representation....23

4.2.4 Dependency....23

4.2.4.1 Try It Yourself....24

4.2.4.2 UML Representation....26

4.2.5 Inheritance....26

4.2.5.1 Try It Yourself....27

4.2.5.2 UML Representation....30

4.2.6 Relationships - Big Picture....30

4.2.7 Check Yourself....31

5. Error Zone - commonly made mistakes.....31

6. Sample code.....32

7. Additional Information (Optional).....32

8. Best Practices.....32

9. Summary or Take Away Points.....32

10. Related Topics.....33

11. References.....33

1. Prerequisites

The reader of this topic is supposed to have basic knowledge of OOP concepts and should also know how to write code in Java for the OOP concepts. Also the reader is supposed to have the basic knowledge about the class ArrayList and its usage. Basic knowledge of UML class diagrams would be helpful but not necessary.

2. Purpose

OOP is the programming methodology where the entire application is seen as lots of objects present in it and which interact among themselves. Because of this interaction a relationship gets established between the objects. As the objects are defined using classes, the same relationship gets established between classes as well.

Any Java application of any kind be it simple or complex, we have lots of objects in it and they are related to each other in multiple ways. The objects interact with each other for completing several tasks hence achieving the functionality of the application. In the coming sections we would see what different relationships can exist between objects and hence between classes.

3. Introduction

In any application each object will have its own role/responsibility along with its characteristics. No object can ever exist isolated and complete a task/functionality of the application. To achieve a goal or functionality, the object has to depend on other objects for further help. This thus creates a relationship between the objects. Depending on the type of dependency between objects, we have different kinds of relationship between objects. The same is represented as relationship between classes.

4. Detailed Explanation

4.1 Class Relationship

Having given a very brief introduction to what relationship between classes is, let us elaborate this term in detail to have further clarity. The coming sections would give an idea of what different relationships can exist between classes and will explore each of the relationship in detail.

Before we jump to the actual topic with technical details, let us try to understand the term relationship with the help of what we see around us in this real physical world.

4.1.1 Relating with Real World Scenario

Each and every day we see lots of objects around us and we see all of them interacting with each other, to perform an action or to complete a goal. Even we take help of different objects to complete our job.

Let us take a small example for understanding the above statement.

Let us consider the working of a school. What do we see in a school?

1. School
2. Class Rooms
3. Principal
4. Teachers, Head Masters
5. Clerk
6. Students
7. Text Books or Study Material
8. Writing Instruments (such as boards, chalk, markers, pen, note books etc)
9. Display instruments (such as boards, TV screen, projector etc.)

For each of the entries mentioned above we have one or more physical entities (we will call them as objects) belonging to that corresponding category. So each entry mentioned above in the list represents a class of objects. So we have One principal object from the Principal class, multiple teacher objects from the Teacher class, multiple pen objects, pencil objects from the Writing instruments class and the list goes on.

NOTE: Whenever the term object/objects is being used in any real world example, it refers to both living being or non-living object (treated equally).

We all know the main goal of a school is to impart learning. With this in mind we have the next question, can any of the objects complete this task/goal without the help of the other objects? The answer for this is definitely NO. A principal takes help of headmasters, teachers for knowledge transfer. The teachers take help of the writing and display instruments, study material to explain the subject. The principal is required for managing the school and the last but not the least, the students are also an essential part of the school. The teachers transfer the knowledge to the students.

With this example, we have understood that each object is dependent on another to perform one or the other task. Now let us try to derive a few relationships among objects from this scenario.

1. School contains the class rooms

2. School has principal, head master, teachers, students
3. Principal, Head Master, Teacher, Clerk are all different specialized types of Staff members in the school.
4. Teachers teach a particular subject by referring to the corresponding text books or study material.
5. Teachers use the different writing or display instruments (that are given to them) for conducting classes. Students also use different writing instruments for taking notes.

Now let us give a name to each of the type of relationship we have identified in the above list

1. Composition
2. Aggregation
3. Inheritance
4. Association
5. Dependency

Let us explain each of the relationship in a few words:

1. Composition: One entity contains another entity. The later entity cannot exist if the former one does not exist.
2. Aggregation: Similar to composition, but both the contained and containing entities can exist independently.
3. Inheritance: One entity is similar to another entity but has some extra special characteristics or roles/responsibilities. (Parent child relationship)
4. Association: An entity takes help of another entity to complete a task.
5. Dependency: An entity is dependent on another entity to complete a task.

NOTE: Both composition and aggregation represent HAS-A relationship and inheritance represents IS-A relationship between two classes.

NOTE: Dependency and association relationships might be confusing and look ambiguous here. While discussing the details, the difference can be seen.

The next section describes the above identified relationships in detail, along with technical details of it.

4.2 Class Relationship Types

With the help of a school example we have identified different relationships that can exist between classes. Whenever we say a class is related to another class, then we would see an interaction between the objects of both the classes.

Except for inheritance relationship there is no special coding that needs to be done to establish the relationship between different classes. Whenever we say any relationship exists between two classes A and B, we can see a reference to an object of the later class and also attributes or a methods of the later class being used in the former class.

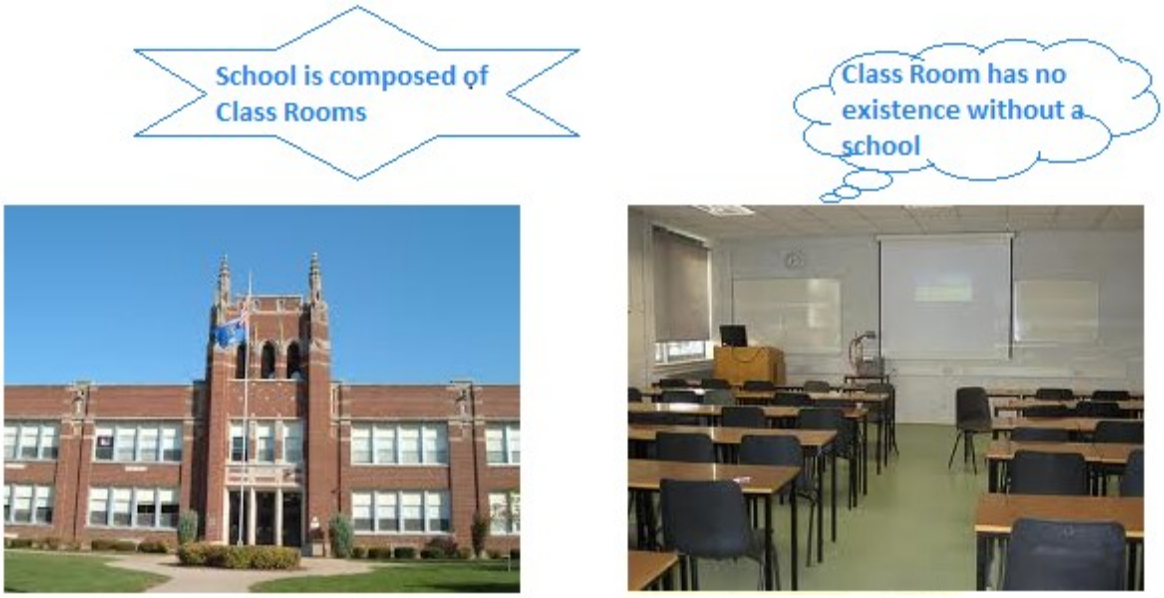
With this understanding let us proceed further with technical details.

4.2.1 Composition

Let us discuss about the first relationship COMPOSITION

The word itself says, one object composes of other objects. Hence the composed objects do not have any existence, if they are taken out separately.

Composition relationship exists between two classes, say A and B, when any of the objects of the class B cannot exist if an object of the class A does not exist. In the above example School contains Class Rooms. A class room cannot exist (or in other terms has no value) if school does not exist. Hence this relationship is termed as composition. On the similar lines, we can take another example for composition is the relationship between school and its address.



Let us see how the composition relationship looks in a Java code.

4.2.1.1 <u>Try It Yourself</u>

Create a class School. It has different attributes such as name, type, date of establishment, rating etc.

Create a class Address. It has different components of address, that is address lines, city, state, country pin code.

Create a class ClassRoom. It has different attributes room number, capacity etc.

As School contains address and some class rooms, a reference of Address and a list of ClassRoom objects also become the attributes of School class. Because address and class room

do not have any specific importance if School does not exist, we say the relationship between School & Address or School & ClassRoom is composition.

The following is the code snippet. The complete example can be found in the zip file.

```java
package com.tcs.ilp.school;

import java.util.ArrayList;

public class School
{
    private String name;
    private String type;
    private Date establishDate;
    private int rating;
    private boolean recognized;

    private Address address; //School has address

    private List<ClassRoom> roomList = new ArrayList<ClassRoom>(); //School has multiple class rooms

    public School() {}


    public School(String name, String type, Date establishDate, int rating,
            boolean recognized, String addrLine1, String addrLine2, String addrLine3,
            String city, String state, String country, long pinCode)
    {
        this.name = name;
        this.type = type;
        this.establishDate = establishDate;
        this.rating = rating;
        this.recognized = recognized;

        this.address = new Address(); //New address object created
        //values set for the address object
        address.setAddrLine1(addrLine1);
        address.setAddrLine2(addrLine2);
        address.setAddrLine3(addrLine3);
        address.setCity (city);
        address.setState(state);
        address.setCountry (country);
        address.setPinCode (pinCode);

    }


    public void setClassRoom(int roomNum, int capacity, int floorNum, boolean acType)
    {
        ClassRoom classRoom  = new ClassRoom(); //Class Room object created
        //values set for the class room object
        classRoom.setRoomNum(roomNum);
        classRoom.setFloorNum(floorNum);
        classRoom.setCapacity(capacity);
        classRoom.setAcType(acType);
        roomList.add(classRoom);
    }


}
```

The getter and setter methods are omitted to save space

```java
package com.tcs.ilp.school;

import java.util.Date;

public class TestComosition
{

    public static void main(String[] args)
    {

        TestComosition testComp = new TestComosition();
        School school = testComp.getSchoolDetails();

        //Set class Room details for the school object
        //Assuming 3 class rooms in the school.
//      The number of rooms can be taken as input from end user to make the program dynamic.
        testComp.setClassRoomDetails(3,school);


        //printing school details
        school.printSchoolDetails();

        //if school object is removed from memory.
        //we do not have access to the address and class room objects present inside school
        school = null;



    }
```

```java
private void setClassRoomDetails(int numRooms, School schoolObj)
{
    Scanner sc  = new Scanner(System.in);

    for(int i=numRooms;i>0;i--)
    {
        //Read class room details
        System.out.println("Please enter room number");
        int roomNum = sc.nextInt();
        System.out.println("Please enter floor number");
        int floorNum = sc.nextInt();
        System.out.println("Please enter capcity of the class room");
        int capacity = sc.nextInt();
        System.out.println("Is the room air conditioned (true/false)");
        boolean ac = sc.nextBoolean();
        schoolObj.setClassRoom(roomNum, capacity, floorNum, ac);
    }
    sc.close();
}
```

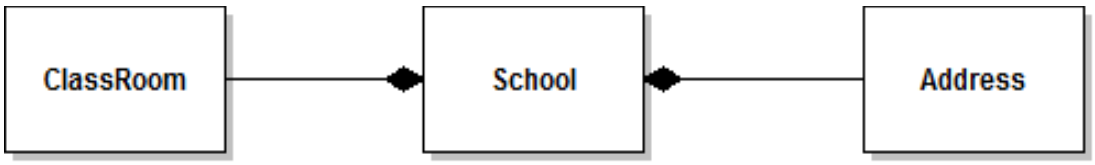
```java
private School getSchoolDetails()
{
    Scanner sc  = new Scanner(System.in);
    //Reading school details from end user
    System.out.println("Please enter the name of your school");
    String schoolName = sc.nextLine();
    System.out.println("Please enter the type of your school (Primary/High)");
    String schoolType = sc.nextLine();
    System.out.println("Please enter the date on which the school was established (DD/MM/YYYY)");
    String strEstablishDate = sc.nextLine();
    Date schoolEstblDate =  DateConverter.convertDate(strEstablishDate);
    System.out.println("Enter rating of your scale");
    int rating = sc.nextInt();
    System.err.println("Is your school recognized by Government (true/false)");
    boolean recg = sc.nextBoolean();


    //Reading address of the school from end user
    System.out.println("Please provide the address of your school");
    System.out.println("Enter Address Line 1 ");
    String addr1 = sc.nextLine();
    System.out.println("Enter Address Line 2");
    String addr2 = sc.nextLine();
    System.out.println("Enter Address Line 3");
    String addr3 = sc.nextLine();
    System.out.println("Enter City");
    String city = sc.nextLine();
    System.out.println("Enter State");
    String state = sc.nextLine();
    System.out.println("Enter Country");
    String country = sc.nextLine();
    System.out.println("Enter pin code");
    int pin = sc.nextInt();

    School school = new School(schoolName, schoolType, schoolEstblDate, rating, recg,
            addr1, addr2, addr3, city, state, country, pin);

    return school;

    }

}
```

Observation: If we observe the code,

- the objects of ClassRoom or Address do not exist if the object School is garbage collected.
- We would not be able to retrieve the objects of ClassRoom or Address once the object of School is removed from memory.

- Another point to note here is that the class room objects or the address object was created by the School class only, so that when object of school is removed from memory automatically others are also garbage collected.
- Here neither the address attribute nor the class room list have getter/setter methods.

4.2.1.2 UML Representation



4.2.2 Aggregation

The next relationship that we would explore is AGGREGATION.

The term aggregation says an object contains other objects.

Aggregation is similar to composition. It also denotes a HAS-A relationship. The only difference between aggregation and Composition is, in an aggregation relationship both the contained and containing objects can exist independent of the other, whereas in composition, the contained object has no existence/importance if the containing object does not exist.

Let us see how aggregation is implemented in Java.

4.2.2.1 Try It Yourself

Take the same School class taken above. Create Teacher class. This class has attributes like employee id, name, subject (assuming a teacher teaches only one subject), experience etc.

The school contains teachers and principal, so we have a list of teachers as an attribute and a principal as another attribute. Here the teachers or the principal can exist even if the school does not exist. Hence here we have the aggregation relationship here.

The following is the code snippet. The complete example can be found in the zip file.

```
package com.tcs.ilp.school;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;

public class School
{
    private String name;
    private String type;
    private Date establishDate;
    private int rating;
    private boolean recognized;

    //Teacher class is referred here
    private List<Teacher> teachers = new ArrayList<Teacher>();


    public List<Teacher> getTeachers() {
        return teachers;
    }


    public void setTeachers(List<Teacher> teachers) {
        this.teachers = teachers;
    }

    public void setTeachers(Teacher teacher) {
        this.teachers.add(teacher);
    }

    public School() {}

    public School(String name, String type, Date establishDate, int rating,
            boolean recognized)
    {
        this.name = name;
        this.type = type;
        this.establishDate = establishDate;
        this.rating = rating;
        this.recognized = recognized;
    }

}
```

The getter and setter methods are omitted to save space

```java
package com.tcs.ilp.school;

import java.util.ArrayList;

public class TestAggregation
{

    public static void main(String[] args)
    {

        TestAggregation testComp = new TestAggregation();
        School school = testComp.getSchoolDetails();

        //Setting the teacherlist with the school
        List<Teacher> teachersList = testComp.getTeacherDetails();
        school.setTeachers(teachersList);

        //printing school details
        school.printSchoolDetails();

        //if school object is removed from memory.
        //we do not have access to the address and class room objects present inside school
        school = null;
        System.out.println("School Removed");
        System.out.println("Now school is ::"+school);
        System.out.println("And the Teachers list is ::: "+teachersList);

    }


    private School getSchoolDetails()
    {
        Scanner sc  = new Scanner(System.in);
        //Reading school details from end user
        System.out.println("Please enter the name of your school");
        String schoolName = sc.nextLine();
        System.out.println("Please enter the type of your school (Primary/High)");
        String schoolType = sc.nextLine();
        System.out.println("Please enter the date on which the school was established (DD/MM/YYYY)");
        String strEstablishDate = sc.nextLine();
        Date schoolEstblDate =  DateConverter.convertDate(strEstablishDate);
        System.out.println("Enter rating of your school (1-5)");
        int rating = sc.nextInt();
        System.err.println("Is your school recognized by Government (true/false)");
        boolean recg = sc.nextBoolean();


        School school = new School(schoolName, schoolType, schoolEstblDate, rating, recg);

        return school;
    }
```

```java
    private List<Teacher> getTeacherDetails()
    {
        //A list holding all the teacher details
        List<Teacher> teacherList = new ArrayList<Teacher>();

        Scanner sc  = new Scanner(System.in);

        //Assuming 5 teachers are there in the school
        for(int i=0; i<5; i++)
        {

            //Getting teacher details from the end user
            System.out.println("Enter Teacher's Name");
            String name = sc.nextLine();
            System.out.println("Enter Teacher's Employee ID");
            int empID = sc.nextInt();
            sc.nextLine();//consume the extra new line character

            System.out.println("Enter the Total Experience in Teaching");
            int experience = sc.nextInt();
            sc.nextLine();//consume the extra new line character

            System.out.println("Enter the Subject the Teacher is Teaching");
            String subject = sc.nextLine();

            Teacher teacher = new Teacher(empID,name,experience,subject);
            teacherList.add(teacher);
        }
```

Teacher object created outside School class

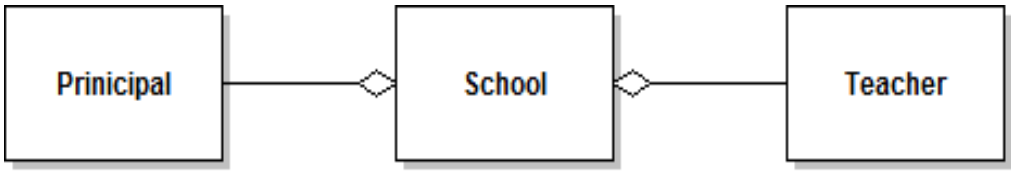
```java
        return teacherList;
    }


}
```

Observation: If we observe the code,

- The school object is created separately and the teacher objects are created separately(outside the School class). Whereas in composition the ClassRoom and Address objects were created inside the School class only
- Later the school object is assigned the list of teacher objects through the setter methods.
- Hence, even if the school object is deleted from the memory, the teachers/principal object can still be accessed.

- The getter/setter methods are available for the teacher/principal objects in the school class.

4.2.2.2 UML Representation



4.2.3 Association

Till now we have discussed about aggregation and composition. Both are similar to each other with a little difference. If there is an aggregation or composition relationship between two classes, then we can say there is an association present between two different classes. The aggregation or composition is also known as whole-part association or HAS-A association. But this is not the only way one class is associated with another.

A class can be associated with another class, if a given object, in its life time knows about the existence of another object and uses its attributes/methods for performing a task, then both the objects are known to be associated with each other and hence the corresponding classes have an association (relationship) between them.

In association, even if one class has a reference to the object of another class, and also uses its attributes or methods for performing certain tasks, it does not take ownership of the objects of the other class. To put in other words, the relationship cannot be logically described as a HAS-A relationship. It can be just described as uses/takes help or such kind of relationship.

NOTE: The association can be uni-directional or bi-directional. Also it can be one-one, one-many, many-one or many-many.

NOTE: If an association can be described as whole-part relationship then it becomes an aggregation. If the whole-part relationship is strong such that part cannot exist without the whole, then the relationship becomes composition.

4.2.3.1 Try It Yourself

Take the same teacher class as in the above example.

The teacher uses some study material for teaching a particular subject. The study material is used by the teacher, for the entire lifetime.

Create a class StudyMaterial with attributes like, author, title, etc.

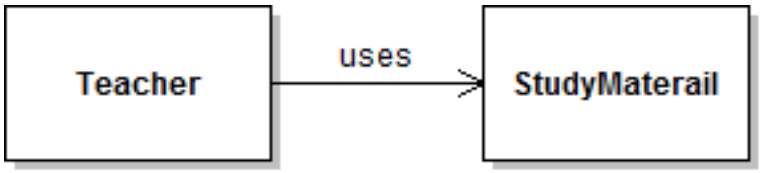
Create an attribute in Teacher class that references object of StudyMaterial. This reference can be used accordingly in methods when ever required.

```java
package com.tcs.ilp.school;

public class Teacher
{

    private int empid;
    private String name;
    private int experience;
    private String subject;

    //Teacher is associated with the study material directly for the entire lifetime
    //The study material can be modified at any point of time or
    //  can be made null but the association is always valid
    private StudyMaterial material;

    public void teach(String topic)
    {

        System.out.println("Teacher is teaching :::"+topic);
        String data = material.retrieveContent(topic);
        System.out.println(data);

    }

}
```

```java
package com.tcs.ilp.school;

public class StudyMaterial
{

    private String author;
    private String title;
    private String type;

    public String retrieveContent(String topic)
    {
        return "The topic retrieved from material :::"+topic;
    }
}
```

Observation: If we observe the code then syntactically there is no difference between association and aggregation. The difference is only conceptual or logical.

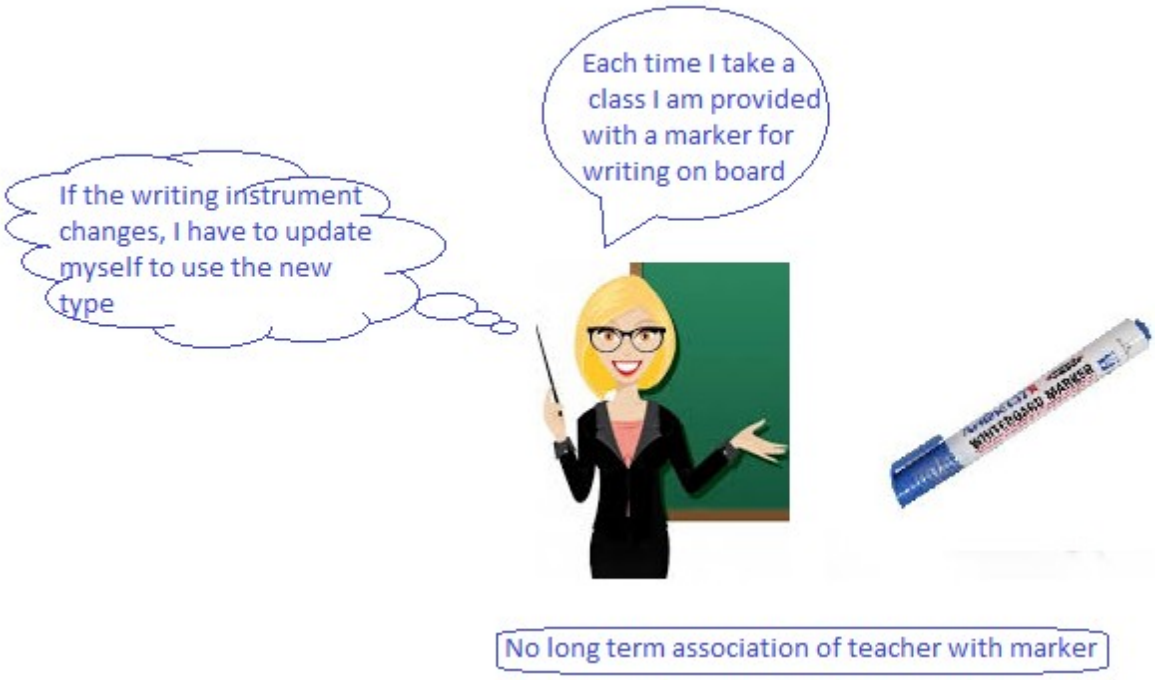
4.2.3.2 UML Representation



4.2.4 Dependency

When a class is associated with another we can say a dependency relationship exists between two classes. Logically both the association and dependency relationships are similar. The difference is only how we code and how strong the relationship is. Association is stronger when compared to the dependency relationship. Association can also be called as a stronger dependency.

The association relationship is valid for the complete lifetime of an object, whereas the dependency relationship is valid only for a limited period of time.



To elaborate the previous statement, when we say association relationship exists between class A and B then, a reference variable for object of class B is found as an attribute in class A. Whereas, when we say class A and B have a dependency relationship, then reference variable for object of class B is found either as a local variable, or as an input parameter to some method of class A.

Let us see the sample code. Complete example is available in the zip file.

4.2.4.1 <u>Try It Yourself</u>

Take the same Teacher class as in the above examples.

Create a class WhiteBoardMarker with attribute color.

An object of WhiteBoardMarker is required by the teacher only when there is a need to write on the board . Hence the object of WhiteBoardMarker is passed as input parameter to method *writeOnBoard* present in the Teacher class. The association between a teacher and a

WhiteBoardMarker object is not present for the entire lifetime of the teacher. Hence this relationship is called dependency.

```java
package com.tcs.ilp.school;

public class Teacher
{

    private int empid;
    private String name;
    private int experience;
    private String subject;


    public void writeOnBoard(WhiteBoardMarker marker, String data)
    {
        //Uses the marker for conducting the classes
        marker.write(data);

        //Next time this method is called the teacher may use a different marker
    }

}
```

```java
package com.tcs.ilp.school;

public class WhiteBoardMarker
{

    public void write(String data)
    {
        System.out.println("Writing the data on white board using the marker");
    }

}
```

Observation: In the code above observe that

- the reference to WhiteBoardMarker is supplied to the teacher during run time.
- There is no long term association of the teacher with the marker object. The teacher object uses the marker object only for a shorter duration.

In contrast to this observe the sample code for association, the teacher object has long term association with the study material object.

Refer the below code snippet to observe the difference.

Dependency:

```
public void writeOnBoard(WhiteBoardMarker marker, String data)
{
    //Uses the marker for conducting the classes
    marker.write(data);

    //Next time this method is called the teacher may use a different marker
}
```

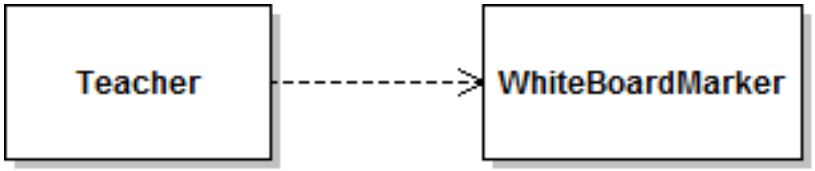
Reference to other class is present as a local variable/input parameter

Association:

```
public class Teacher
{
   ..................................
   private StudyMaterial material;

}
```

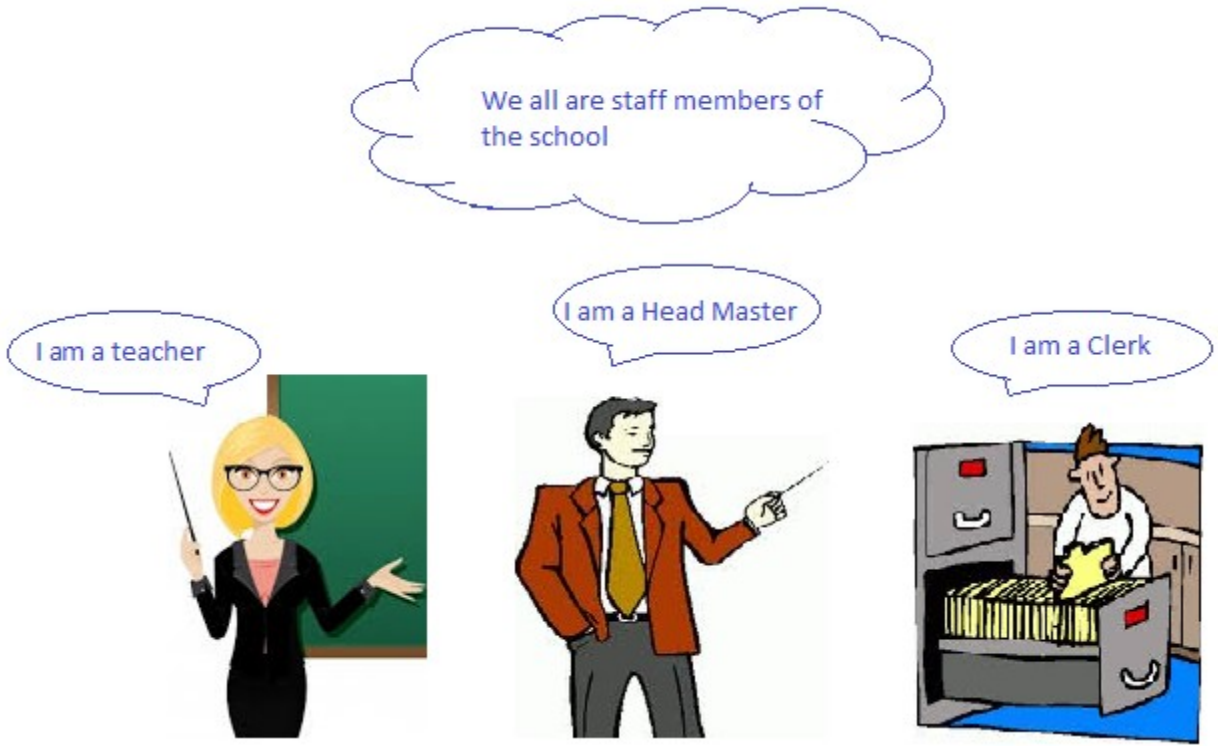
Reference to other class is present as an attribute of class

4.2.4.2 UML Representation



4.2.5 Inheritance

The last relationship that we are going to discuss here is the inheritance relationship. This kind of relationship exists between two classes when one class is a specialized type of another. In a school we have the teachers, headmasters, principal, clerk etc. All of them can be generalized as school staff. As school staff they have some common attributes such as employee id, name salary and some common tasks, such as signing staff attendance register etc. Again each of them have their additional characteristics and their own specialized task such as the teacher teaches, the headmaster conducts review, the clerk maintains the attendance registers etc.



Inheritance provides a lot of re-usability. The inheritance feature in Java is used to establish the IS-A relationship.

Let us see sample code in Java. The entire example is present in the zip file.

4.2.5.1 Try It Yourself

Create a class Staff with attributes employee id and name. These are the common attributes for all the staff members of the school. Write a method signRegister. This represents the responsibility of signing the staff attendance register which is a common responsibility for all the staff members.

Create a class Teacher and extend it from Staff. Employee id and name are inherited from the staff class. The attributes experience, subject, study material etc are specific only to this class. Similarly the teach method is specific to the Teacher class and signRegister method is inherited from the Staff class.

Create a new class Clerk and extend it from Staff. Similar to the class Teacher, Clerk also inherits the attributes and methods from Staff class and has its own specialized method maintainRegister for maintaining the register.

```java
package com.tcs.ilp.school;

public class Staff
{

    protected int empid;
    protected String name;


    public int getEmpid() {
        return empid;
    }
    public void setEmpid(int empid) {
        this.empid = empid;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }


    public void signRegister()
    {
        //Every staff member of the school
        //signs in the attendance reigster
        System.out.println(name+" signed in the attendance register");
    }

}
```

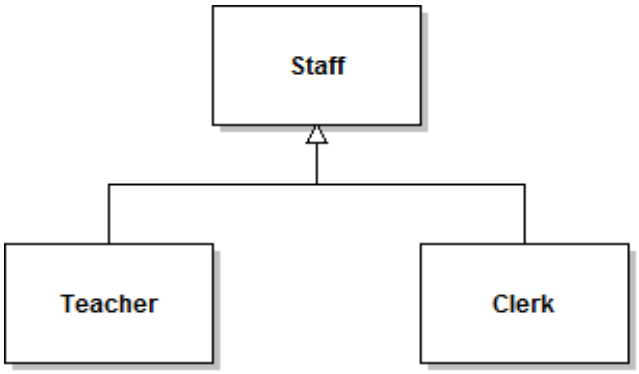
```java
package com.tcs.ilp.school;

public class Clerk extends Staff
{

    //Additional functionality for the clerk
    //is to maintain the registers
    public void maintainRegister()
    {
            System.out.println(name+" maintains the attendance registers");
    }


}
```

```java
package com.tcs.ilp.school;

public class Teacher extends Staff
{

    //name and empid inherited from class Staff

    //additional characteristics of a teacher
    protected int experience;
    protected String subject;

    //Teacher is associated with the study material directly for the entire lifetime
    //The study material can be modified at any point of time or
    //  can be made null but the association is always valid
    protected StudyMaterial material;

    public void teach(String topic)
    {
        System.out.println(name+" is teaching :::"+topic);
        String data = material.retrieveContent(topic);
        System.out.println(data);

    }


}
```

Observation: In the code above observe that

- the Clerk and the Teacher inherit the Staff class and hence inherit the methods and attributes of the Staff class.
- The ***extends*** keyword shows the inheritance relationship between Clerk – Staff and Teacher – Staff.
- The Clerk has the extra responsibility of managing the attendance registers. This is done with the help of *maintainRegister* method in the Clerk class.
- Whereas the Teacher has the additional responsibility of teaching. This is accomplished with the *teach* method in the Teacher class.

4.2.5.2 UML Representation



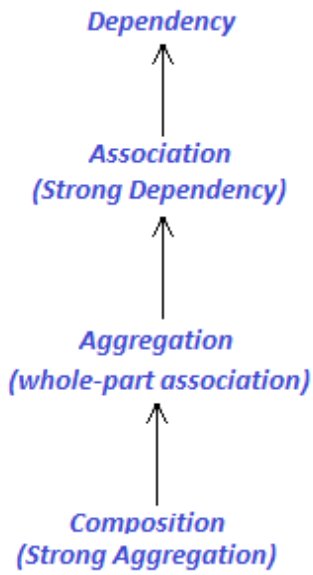
4.2.6 Relationships - Big Picture

To summarize the relationships which we have discussed till now can be categorized as follows:

1. Any kind of relationship between two classes can be termed as Dependency relationship.
2. If the identified dependency is strong enough, such that one class has an attribute which refers to the other class, then the relationship will be termed as Association.
3. If the identified association represents a logical whole-part relationship or in other words HAS-A relationship, then the relationship will be called as Aggregation.

4. If the whole-part relationship is strong enough such that, the object representing the part cannot exist without the existence of the object representing the whole then the relationship will be called as Composition.
5. If one class is a specialized form of another class then the relationship between the two would be termed as Inheritance.

The above discussion can be represented in the form of a diagram as below:



4.2.7 Check Yourself

1. What are the different relationships between classes?
2. What is the difference between aggregation and composition?
3. What is the difference between association and dependency?
4. Are aggregation and composition special type of association?
5. What is the other name for aggregation and composition relationship?
6. What is the other name for inheritance relationship?
7. Is association a type of dependency?

5. Error Zone - commonly made mistakes

The beginners tend to over use inheritance in the code. This might not look like a problem for a small example code. But in huge applications more use of inheritance tends to make the code less modifiable and hence maintenance becomes tough.

When two entities (classes) differ only with respect to the characteristics or attributes then aggregation or composition should be preferred. Whereas inheritance should be used only when there are some specialized responsibilities/tasks (methods) that have to be assigned to an entity (or class) along with some of the general characteristics and responsibilities which could be inherited from another entity (or class).

6. Sample code

7. Additional Information (Optional)

While drawing class diagrams another relationship, known as realization is also shown. This relationship is depicted when a relationship between interface and its implementing class is shown.

The example codes discussed can be further extended to have an interface For example,we can have an interface Writable and all the writing instruments can implement that interface. The same concept can be applied for display instruments.

8. Best Practices

The same re-usability that inheritance provides, can also be obtained by using aggregation. It is a good practice to make use of HAS-A relationship, wherever possible, instead of IS-A relationship. This ensures that we do not end up in maintenance problems.

9. Summary or Take Away Points

Till now we have discussed.

- Objects interact with each other to perform tasks and hence are related to each other. This leads to relationship between classes.
- Different types of possible relationship between classes are
  - Association
  - Aggregation
  - Composition

- Dependency
- Inheritance

- Association is a strong type of dependency
- Aggregation is nothing but HAS-A association between two classes.
- Composition is also a HAS-A association and is special type of aggregation
- Inheritance is also known as IS-A relationship.
- Prefer HAS-A relationship than IS-A relationship.

10. Related Topics

- UML class diagrams.
- Inheritance.

11. References

- http://javapapers.com/oops/association-aggregation-composition-abstraction-generalization-realization-dependency/
- https://www.google.co.in/url?sa=t&rct=j&q=&esrc=s&source=web&cd=9&sqi=2&ved=0CFIQFjAI&url=http%3A%2F%2Fwww.ldodds.com%2Flectures%2Fintro2java%2FUML_Examples.doc&ei=hqiCU8yVKYWHuASz84DIBQ&usg=AFQjCNG_JeaHyHiY_Ka8aFtTGtcr8HFC_Q&bvm=bv.67720277,d.c2E&cad=rja